

4	TLO 4.1 Compare fixed and variable memory partitioning. TLO 4.2 Differentiate between Bit map and Linked list technique. TLO 4.3 Explain working of various partitioning algorithm. TLO 4.4 Calculate page fault for given page reference string.	Unit - IV Memory Management 4.1 Basic Memory Management: Partitioning - Fixed and Variable, Free Space Management Techniques: Bit map, Linked List 4.2 Swapping, Compaction, Fragmentation, Partitioning Algorithms: First fit, Best fit, Worst fit 4.3 Non-contiguous Memory Management Techniques: Paging, Segmentation 4.4 Virtual Memory: Basics, Demand paging, Page Fault 4.5 Page Replacement Algorithm: First In First Out (FIFO), Least Recently Used (LRU), Optimal	Lecture Using Chalk-Board Presentations Video Demonstrations
---	--	---	---

Free Space Management Techniques (6 marks) (4 marks)

Free space management (02)

A file system is responsible to allocate free blocks to the file therefore it has to keep track of all the free blocks present in disk.

Free space management is critical aspect of operating system as it involves managing the available storage space on the hard disk or secondary storage devices.

The operating system uses various techniques to manage free space and optimize the use of storage device.

List of free space management techniques. (02)

- ✓ Bit vector or Bitmap
- ✓ Linked List
 - Grouping
 - Counting

1) Bit vector or Bitmap (02)

- The free space list is implemented as bitmap or bit vector.
- It is series of collection of bits where each bit corresponds to disk block.
- The bit can take two value 0 & 1.
- If block is free 0 & if block is allocated

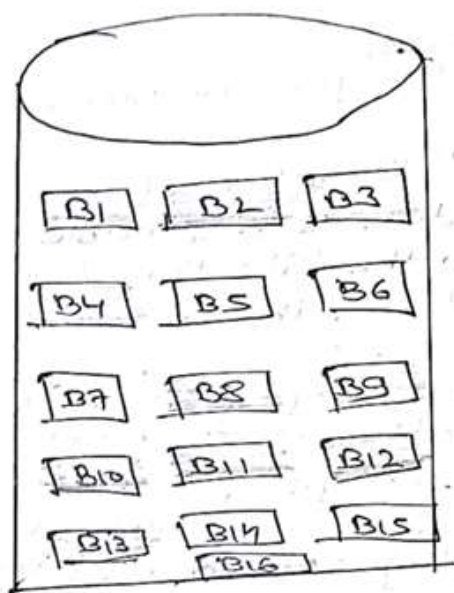
1

For example,

The given instance of disk blocks on disk in Figure, which are colored are allocated in Figure. which are colored are allocated can be represented by 111100011111001

Advantages

- Easy to understand & implement
- Efficient to find first free block by scanning the non-zero word, then identifying first bit



B1/B2... B16
Blocks

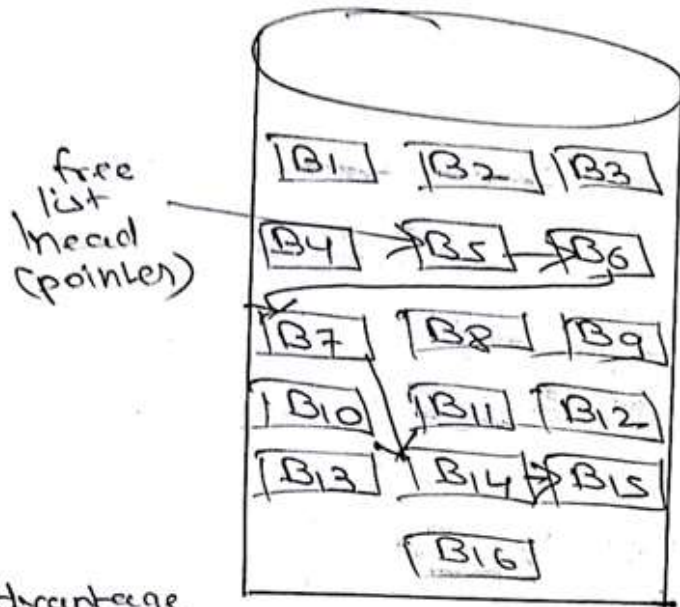
☒ Allocated
☐ free

Disadvantage

- finding free blocks may become slower as it requires scanning of blocks
- more memory space if the storage or disk is large

2) Linked-List (02)

- In this approach, the free disk blocks are linked together i.e. a free block contains pointers to next free block.
- The block number of very first disk block is stored at separate location on disk and is also cached in memory.
- In this approach, link all the disk blocks together keeping a pointer to first free block.
- This block contains a pointer to next free disk block and so on.



Advantage

- o Total available space is efficiently used in method.
- o Dynamic allocation in Linked List is easy.

DisAdvantage

- o As Linked List size increases, headache of memorizing pointers increases.
- o Not efficient while traversal of each block.

Introduction

Memory management

Memory management is how the operating system controls and divides memory between different programs running on computer.

It ensures that the main memory is used efficiently and helps manage the transfer of data between the memory and disk while programs are running.

Functions of memory management

- 1) Keep tracking of every memory location.
- 2) Tracks of whether memory is utilized properly.
- 3) Tracks how much memory is allocated.
- 4) Tracks of whether memory is allocated or not.
- 5) Takes decision which process will get memory and when.

Partitioning in memory management

Partitioning in main memory management is how a computer's memory is divided to run multiple programs.

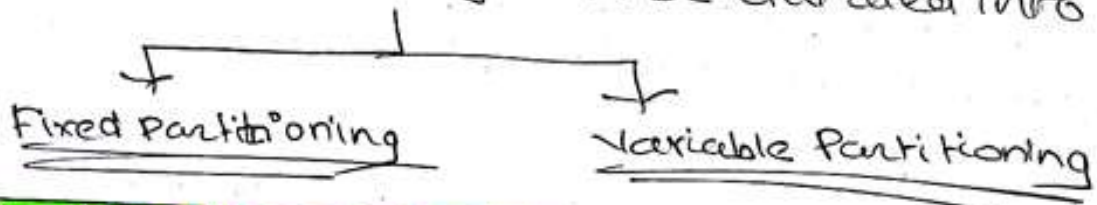
The main memory (RAM) is split into sections, and each section holds one program.

There are two methods

- ↳ Non-Contiguous memory allocation.
- ↳ Contiguous memory allocation

Contiguous memory allocation

- Each program gets one big, continuous block of memory.
- The entire program must be in memory to run.
- Contiguous memory can be divided into.



Fixed Partitioning (Static)

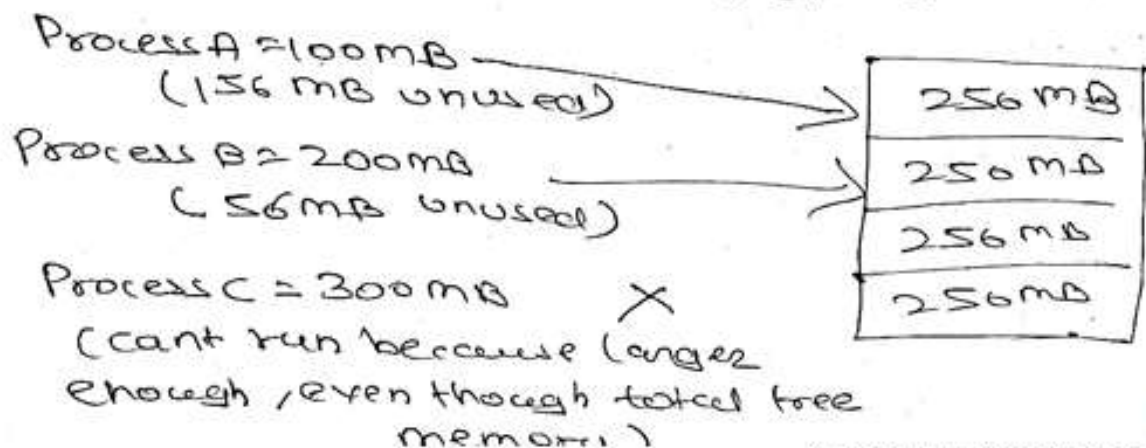
Fixed partitioning divides memory into a set number of fixed-size blocks.

Each block holds one process, and the partition sizes are set when system starts and do not change.

Key points

- Each partition has a fixed size.
- If a process doesn't use the entire partition, the unused space is wasted (called internal fragmentation).
- The number of partitions limits how many processes can run at same time.

Example computer has 1024 MB divided into 256 MB (4 part)



Variable Partitioning

Variable Partitioning allocate memory dynamically based on exact amount a process needs.

Unlike fixed partitioning, the memory is not divided into fixed blocks. Instead the system creates partitions as processes arrive and frees them when they finish.

Key points :

- Partition size vary depending on the process need.
- memory is used more efficiently because it allocate exactly what's required.
- No internal fragmentation, but external fragmentation can happen when small gaps form as processes finish and leaves holes in memory.

Example: System (1024 MB Ram)

Process X (100 MB) allocated, 924 MB free
Process Y (300 MB) allocated, 624 MB free
Process Z (150 MB) allocated, 474 MB free

If Process Y finishes and free 300 MB, the memory split into two parts:
300 MB of free space & 174 MB elsewhere.

If new process W needs 380 MB
it can't be allocated because available memory separate smaller blocks, not a single large block.

This is external fragmentation

4.3 Non-contiguous Memory Management Techniques: Paging, Segmentation

Virtual memory

↳ Virtual memory lets computer use part of its harddrive to act like extra RAM when physical (RAM) is full.

↳ It moves data temporarily from RAM to disk storage to free up space.

↳ Programmers have not to worry about how much RAM the computer has, as virtual memory makes it seem like there's more memory available.

↳ This makes programming easier since developers can focus on coding without worrying about memory limits.

iii) Memory compaction

Paging

Paging is memory management scheme that permits the physical address space of process to be non contiguous.

Logical memory is divided into blocks, same size called as pages.

Paging is a storage mechanism used in OS to retrieve processes from secondary storage to main memory as pages.

The process of retrieving processes in form of pages from secondary storage into main memory is known as paging.

Segmentation

Segmentation is memory management scheme that permits dividing logical address space into multiple segments.

Segmentation divides the computer's physical memory and program address space into segments.

f) Paging

Paging is a storage mechanism which ~~retrieval~~ retrieval the process from Secondary memory to main-memory at pages.

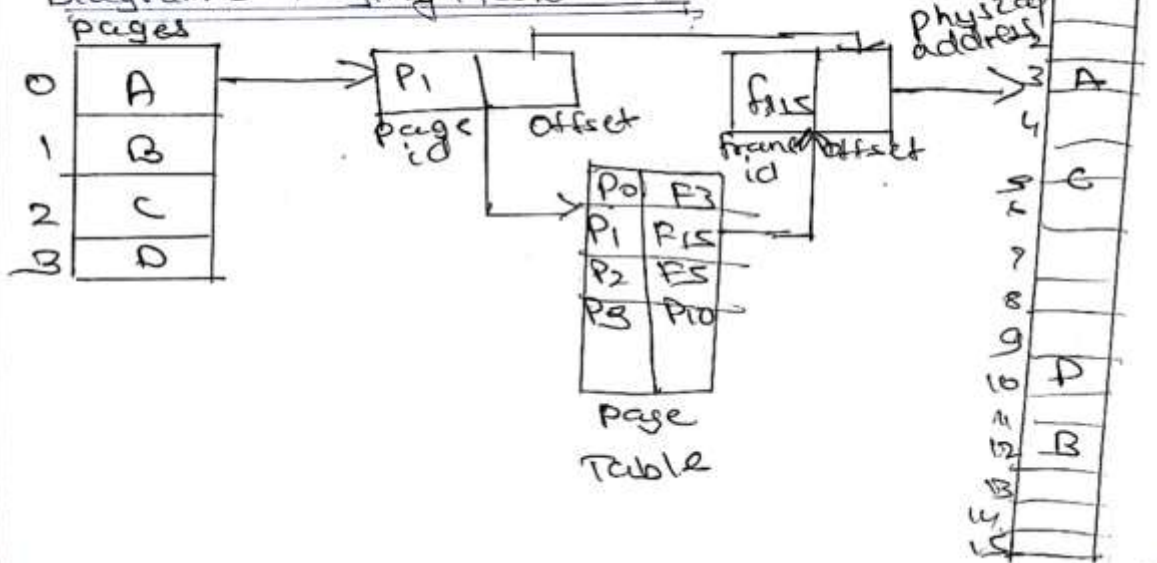
Process of retrieving ~~page~~ memory from it in form of pages from Secondary memory to main memory is called as Paging.
Paging is fixed in size.

Homework :

Describe the concept of virtual memory with respect to paging. Also draw paging hardware diagram and describe its working with example. -- 6m.

allow storage to RAM, allowing to continue smoothly without worrying about limited physical memory.

Diagram of Paging hardware.



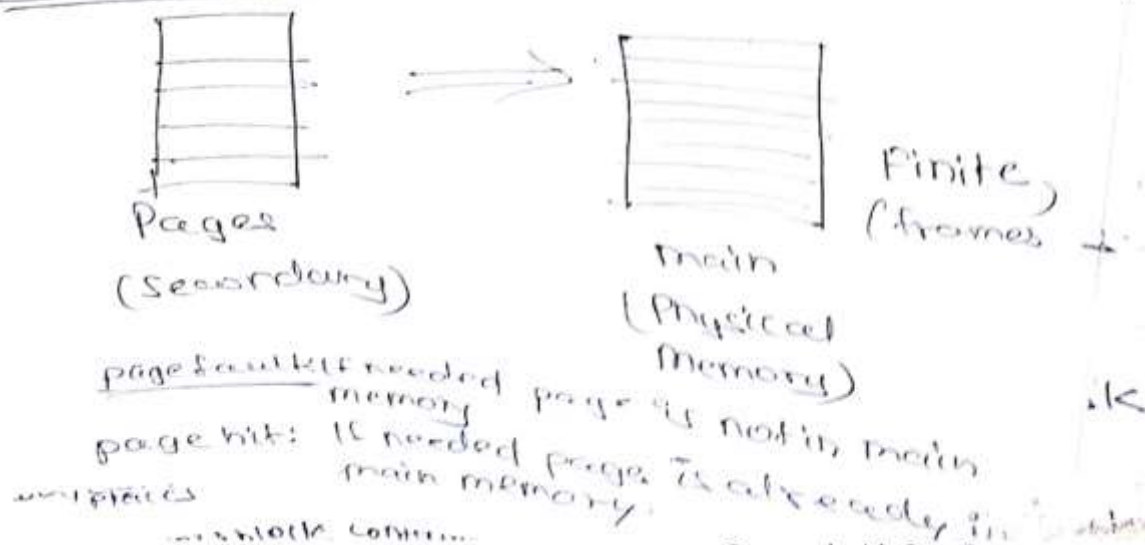
<u>Parameters</u>	<u>Paging</u>	<u>Segmentation</u>
<u>memory structure</u>	Divides the process address space into <u>fixed size</u> pages.	Divides the process address space into <u>variable-size</u> segments.
<u>memory size</u>	Pages are of fixed size.	Segment are of pages varying size.
<u>Speed</u>	Paging is faster in accessing memory.	Segmentation is slower due to variable-size of segments.
<u>Fragmentation</u>	Paging can lead to <u>internal fragmentation</u> if a page has unused space.	Segmentation can lead to <u>external fragmentation</u> when memory gaps appear bet ⁿ segments.
<u>Accountability</u>	The operating system manages memory by dividing into pages.	Compiler divides memory into segments and assign actual or virtual address.
<u>Flexibility</u>	Same size so limited flexibility.	more size ^{flexibility} segments can be different in size.
<u>Data storage</u>	Page tables store the mapping of page to memory.	Segment tables store the mapping of segment to memory.
<u>Logical Address</u>	A logical address is divided into page number and offset.	A logical address is divided into a segment number and offset.
<u>Size Determination</u>	Size is determined by available memory and is fixed for all pages.	The user or program defines segments size which can vary.

page fault

A Page fault is typically occurs when a process attempt to access memory in virtual address space that it does not own.

Page fault is the condition in which a running process refers to page that is not loaded in memory (main).

Demand Paging



Scanned with CamScanner

Question : Define fragmentation. Explain its types. (4m)

Fragmentation

Fragmentation is a problem in operating system where memory is used inefficiently leaving small gaps that cannot be used.

This leads to wasted memory space, making it difficult to allocate memory for new processes even where there is some memory available.

There are two types of Fragmentation

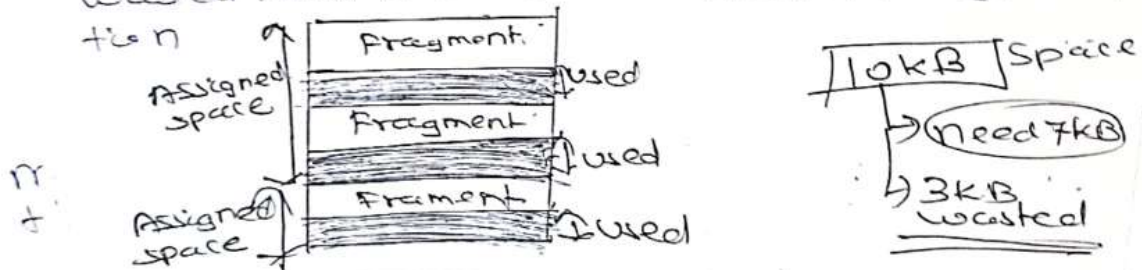
Internal Fragmentation

External Fragmentation

Internal Fragmentation

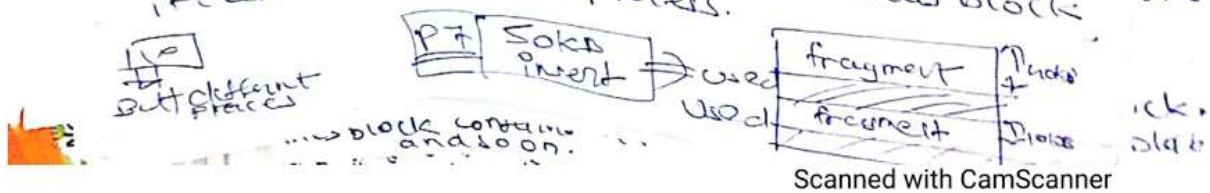
Internal Fragmentations happen when a memory is divided into fixed-size blocks, but a process doesn't need all the space in block.

The left-over space inside the block is wasted, and this waste is Internal Fragmentation.



External Fragmentation

External fragmentation when there is enough total memory to meet the needs of process, but scattered in small pieces. Since the memory is not in continuous block, it can't be used by process.



Homework : compare internal and external fragmentation

c)

Explain the following terms with respect to memory management:

- i) Dynamic relocation
- ii) Swapping

Ans.

i) Dynamic Relocation

When a program gets swapped out to a disk memory, then it is not always possible that when it is swapped back into main memory then it occupies the previous memory location, since the location may still be occupied by another process. We may need to relocate the process to a different area of memory. Thus there is a possibility that program may be moved in main memory due to swapping.

ii) Swapping

Swapping is mechanism in which a process can be swapped temporarily out of main memory (or move) to secondary storage (disk) and make that memory available to other processes. At some later time, the system swaps back the process from the secondary storage to main memory.

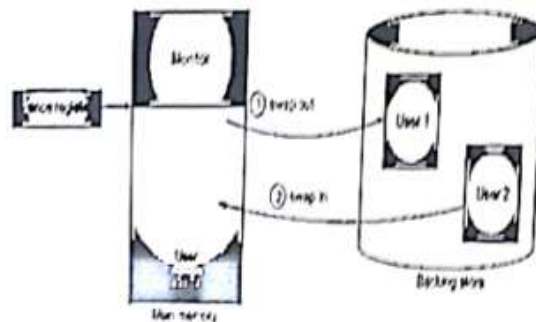


Fig: swapping in memory management

Each ter
explanati
2M

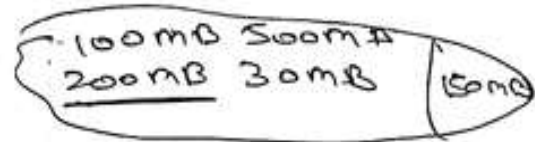
1) First Fit, Best Fit & Worst Fit are methods used by operating system to allocate memory to processes

1) First Fit

Chooses the first available memory block that is big enough for process.

In simple, takes the first memory block big enough to process.

Example, process needs 200MB, available blocks are 300MB & 400MB, it takes 300MB block.



2) Best Fit

It choose the smallest block that can fit the process.

Example, Process need: 200MB.

Blocks are 250MB & 400MB,

It takes 250MB

3) Worst Fit

largest block.

200MB (need)

250 & 500

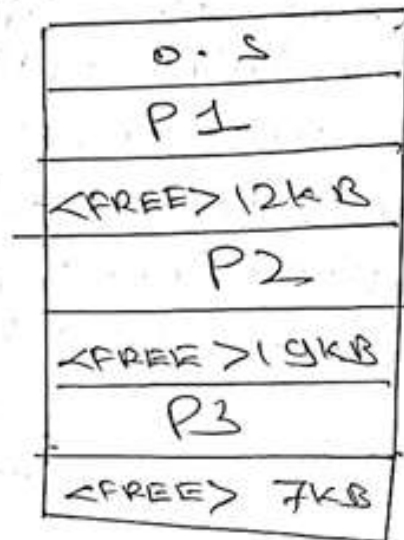
500MB block taken

Questions

Consider the following memory map and assume a new process P4 comes with memory requirements of 6 KB.

Locate (Draw) this process in memory

- i) First Fit
- ii) Best Fit
- iii) Worst Fit



memory

— 6 marks —



<u>First Fit</u>	<u>Best Fit</u>	<u>Worst Fit</u>																					
Allocate the first free block to new process P4.	Allocate smallest free block that is big enough to accomadate new process P4.	Allocate longest free block to new process P4																					
<table><tr><td>OS</td></tr><tr><td>P1</td></tr><tr><td>P4 6KB <FREE> 6KB</td></tr><tr><td>P2</td></tr><tr><td><FREE> 19KB</td></tr><tr><td>P3</td></tr><tr><td><FREE> 7KB</td></tr></table>	OS	P1	P4 6KB <FREE> 6KB	P2	<FREE> 19KB	P3	<FREE> 7KB	<table><tr><td>OS</td></tr><tr><td>P1</td></tr><tr><td><FREE> 12KB</td></tr><tr><td>P2</td></tr><tr><td><FREE> 19KB</td></tr><tr><td>P3</td></tr><tr><td>P4 6KB <FREE> 1KB</td></tr></table>	OS	P1	<FREE> 12KB	P2	<FREE> 19KB	P3	P4 6KB <FREE> 1KB	<table><tr><td>OS</td></tr><tr><td>P1</td></tr><tr><td><FREE> 12KB</td></tr><tr><td>P2</td></tr><tr><td>P4 6KB <FREE> 13KB</td></tr><tr><td>P3</td></tr><tr><td><FREE> 7KB</td></tr></table>	OS	P1	<FREE> 12KB	P2	P4 6KB <FREE> 13KB	P3	<FREE> 7KB
OS																							
P1																							
P4 6KB <FREE> 6KB																							
P2																							
<FREE> 19KB																							
P3																							
<FREE> 7KB																							
OS																							
P1																							
<FREE> 12KB																							
P2																							
<FREE> 19KB																							
P3																							
P4 6KB <FREE> 1KB																							
OS																							
P1																							
<FREE> 12KB																							
P2																							
P4 6KB <FREE> 13KB																							
P3																							
<FREE> 7KB																							

— 02 —

— 02 —

— 02 —

FIFO

Frame size = 3 3 2 1 8 4 1 6 2 4 3 4 2 1 4 5 2 1 3 4

Page	3	2	1	3	4	1	6	2	4	3	4	2	1	4	5	2	1	3	4
f1	3	3	3	3	4	4	4	4	4	3	3	3	3	3	5	5	5	5	4
f2		2	2	2	2	6	6	6	6	6	4	4	4	4	4	2	2	2	2
f3			1	1	1	1	1	2	2	2	2	2	1	1	1	1	1	3	3
Result	fault	f	f	Hit	f	Hit	f	f	Hit	f	Hit	f	f	Hit	f	f	Hit	f	4

page fault = 13 = $\frac{13}{19}$

1894 = $\frac{6}{19}$

Page	5	5	6	7	8	9	7	8	9	5	8	7	8	9	5	7	8	9	5
f1	5	5	5	5	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8
f2			6	6	6	9	9	9	9	9	9	9	9	9	9	9	9	9	9
f3				7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7
Result	f	f	f	f	f	f	f	f	f	f	f	f	f	f	f	f	f	f	Hit

page fault = 11 $\frac{11}{19}$

Page Algorithm of Replacement

[illegible]

page fault = 16

[illegible]

page count = 15.

Page	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100
Page	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100

Page fault = 12

Page	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100
Page	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100

Page fault = 25

and Optimal Page Replacement Algorithm:
 Replace page that will be used not in near future.

page	3	2	1	3	4	1	6	2	4	3	4	2	1	4	5	2	1	3	4
f1	3	3	3	3	4	4	4	4	4	4	4	4	4	4	5	5	5	5	5
f2		2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	3	3
f3			1	1	1	1	6	6	6	3	3	3	1	1	1	1	1	3	4
Page fault	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F

page fault = 10

page	5	6	5	7	8	7	8	9	9	5	5	8	7	9	9	8	9	5	4
f1	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5
f2		6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6
f3																			
Result	f	f	f	f	f	f	f	f	f	f	f	f	f	f	f	f	f	f	f

page fault = 09

Page	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
f1	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7
f2		0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0	0
f3			1	1	1	3	2	3	3	3	3	3	3	1	1	1	1	1	1	1
Result	F	F	F	F	F	F	F	F	F	H	F	H	F	F	H	H	H	F	H	F

Page fault=09

Page	4	1	2	3	1	4	1	5	2	3	4	2	1	3	4	1	2	5	1	2
f1	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	2	2	2	2
f2		1	1	1	1	1	1	5	2	2	2	2	1	1	1	1	1	1	1	1
f3			2	3	3	3	3	3	3	3	3	3	3	3	3	3	3	5	5	5
Result	F	F	F	F	H	H	H	F	F	H	H	H	F	H	H	H	H	H	H	H

Page fault=09